

Fused Multiply Adder

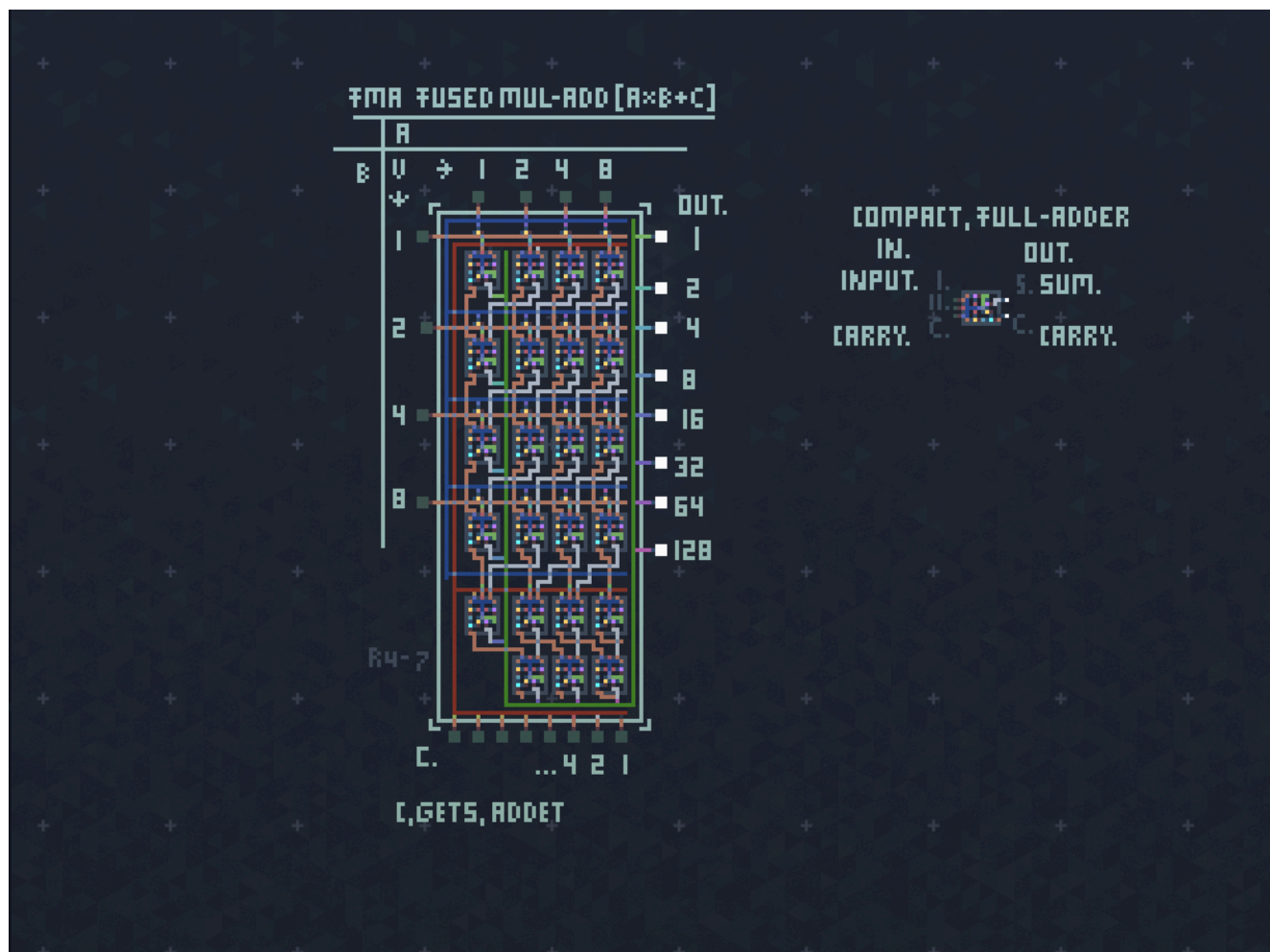
Fused Multiply-Adder (FMA) Module

Concept Overview A **Fused Multiply-Adder (FMA)** is a specialized hardware component designed to compute the operation

$$A \times B + C$$

in a single, unified step.

Instead of routing data through a separate multiplier and then an adder—which wastes space, introduces latency, and compounds rounding errors—the FMA "fuses" these steps. This makes it incredibly efficient for graphics processing unit (GPU) architectures, where matrix multiplication and vector math happen constantly, allowing us to bypass heavy, complex CPU-like control logic.



Left Fused Multitply Adder | Right Compact Full Adder

Pin Assignment & Bus Architecture

The module utilizes a color-coded bus system to make routing and bit-ordering highly modular and adaptable.

Direction	Port	Data Width	Bus Color	Flexibility
TOP	Input A (Factor 1)	4-Bit (A3 ...A0)	 Blue Bus	High (Easily reordered via bus colors)
LEFT	Input B (Factor 2)	4-Bit (B3 ...B0)	 Direct/Orange	Medium (Hardwired; potential upgrade to a bus)
BOTTOM	Input C (Addend)	8-Bit (C7 ...C0)	 Red Bus	High (Easily reordered via bus colors)
RIGHT	Output R (Result)	8-Bit (R7 ...R0)	 Green Bus	High (Easily reordered via bus colors)

Export to Sheets

 **Architecture Upgrade Idea** The **Input B** lines (Orange) are currently direct connections. To maximize modularity, these can easily be converted into their own bus system just like Inputs A and C, allowing for effortless reordering of the B-bits.

Step-by-Step Matrix Calculations

The matrix consists of an array of AND gates fused into Full Adders (FA). *Notation:*

$FA(\text{Input1}, \text{Input2}, \text{CarryIn}) = [\text{Sum}, \text{CarryOut}]$

Row 0 (Top: Multiplication by B0)

Fuses the product of $A \times B_0$ with the lower bits of C.

- **Cell (A0, B0):**
 - $AND_{00} = A_0 \text{ AND } B_0$
 - $FA_{00} = FA(AND_{00}, C_0, 0)$
 -  **Result:** $R_0 = Sum_{00}$ (Direct to Green Output Bus)
 -  **Carry:** $Carry_{00}$ (Passes down to cell A0, B1)
- **Cell (A1, B0):**
 - $AND_{10} = A_1 \text{ AND } B_0$

- $FA_{10} = FA(AND_{10}, C1, 0)$
 - ➡ **Outputs:** Sum_{10} (To A0, B1), $Carry_{10}$ (To A1, B1)
 - **Cell (A2, B0):**
 - $AND_{20} = A2 \text{ AND } B0$
 - $FA_{20} = FA(AND_{20}, C2, 0)$
 - ➡ **Outputs:** Sum_{20} (To A1, B1), $Carry_{20}$ (To A2, B1)
 - **Cell (A3, B0):**
 - $AND_{30} = A3 \text{ AND } B0$
 - $FA_{30} = FA(AND_{30}, C3, 0)$
 - ➡ **Outputs:** Sum_{30} (To A2, B1), $Carry_{30}$ (To A3, B1)
-

Row 1 (Upper Middle: Multiplication by B1)

Catches the intermediate sums and carries from Row 0.

- **Cell (A0, B1):**
 - $AND_{01} = A0 \text{ AND } B1$
 - $FA_{01} = FA(AND_{01}, Sum_{10}, Carry_{00})$
 - ➡ **Result:** $R1 = Sum_{01}$ (Direct to Green Output Bus)
 - ➡ **Carry:** $Carry_{01}$ (Passes down to cell A0, B2)
 - **Cell (A1, B1):**
 - $AND_{11} = A1 \text{ AND } B1$
 - $FA_{11} = FA(AND_{11}, Sum_{20}, Carry_{10})$
 - ➡ **Outputs:** Sum_{11} (To A0, B2), $Carry_{11}$ (To A1, B2)
 - **Cell (A2, B1):**
 - $AND_{21} = A2 \text{ AND } B1$
 - $FA_{21} = FA(AND_{21}, Sum_{30}, Carry_{20})$
 - ➡ **Outputs:** Sum_{21} (To A1, B2), $Carry_{21}$ (To A2, B2)
 - **Cell (A3, B1):**
 - $AND_{31} = A3 \text{ AND } B1$
 - $FA_{31} = FA(AND_{31}, 0, Carry_{30})$ (No incoming partial sum here; set to 0)
 - ➡ **Outputs:** Sum_{31} (To A2, B2), $Carry_{31}$ (To A3, B2)
-

Row 2 (Lower Middle: Multiplication by B2)

- **Cell (A0, B2):**
 - $AND_{02} = A0 \text{ AND } B2$

- $FA_02 = FA(AND_02, Sum_11, Carry_01)$
 - ➡ **Result:** $R2 = Sum_02$ (*Direct to Green Output Bus*)
 - ➡ **Carry:** $Carry_02$ (*Passes down to A0, B3*)
 - **Cell (A1, B2):**
 - $AND_12 = A1 \text{ AND } B2$
 - $FA_12 = FA(AND_12, Sum_21, Carry_11)$
 - ➡ **Outputs:** Sum_12 (*To A0, B3*), $Carry_12$ (*To A1, B3*)
 - **Cell (A2, B2):**
 - $AND_22 = A2 \text{ AND } B2$
 - $FA_22 = FA(AND_22, Sum_31, Carry_21)$
 - ➡ **Outputs:** Sum_22 (*To A1, B3*), $Carry_22$ (*To A2, B3*)
 - **Cell (A3, B2):**
 - $AND_32 = A3 \text{ AND } B2$
 - $FA_32 = FA(AND_32, 0, Carry_31)$
 - ➡ **Outputs:** Sum_32 (*To A2, B3*), $Carry_32$ (*To A3, B3*)
-

Row 3 (Bottom: Multiplication by B3)

- **Cell (A0, B3):**
 - $AND_03 = A0 \text{ AND } B3$
 - $FA_03 = FA(AND_03, Sum_12, Carry_02)$
 - ➡ **Result:** $R3 = Sum_03$ (*Direct to Green Output Bus*)
 - ➡ **Carry:** $Carry_03$ (*Feeds into the final vector adder chain*)
 - **Cell (A1, B3):**
 - $AND_13 = A1 \text{ AND } B3$
 - $FA_13 = FA(AND_13, Sum_22, Carry_12)$
 - ➡ **Outputs:** Sum_13 , $Carry_13$ (*Both pass to the final chain*)
 - **Cell (A2, B3):**
 - $AND_23 = A2 \text{ AND } B3$
 - $FA_23 = FA(AND_23, Sum_32, Carry_22)$
 - ➡ **Outputs:** Sum_23 , $Carry_23$ (*Both pass to the final chain*)
 - **Cell (A3, B3):**
 - $AND_33 = A3 \text{ AND } B3$
 - $FA_33 = FA(AND_33, 0, Carry_32)$
 - ➡ **Outputs:** Sum_33 , $Carry_33$ (*Both pass to the final chain*)
-

88 The Final Adder Chain (For High Bits R4 to R7)

At the bottom left, the remaining accumulated sums and carries drop out of the matrix. We process them through a final ripple-like chain while injecting the remaining upper bits of Input C (C4 to C7).

- **For R4:** $FA_{R4} = FA(\text{Sum}_{13}, \text{Carry}_{03}, C4)$ → $R4 = \text{Sum}, \text{Carry}_{R4}$
- **For R5:** $FA_{R5} = FA(\text{Sum}_{23}, \text{Carry}_{13}, \text{Carry}_{R4} + C5)$
- **For R6:** $FA_{R6} = FA(\text{Sum}_{33}, \text{Carry}_{23}, \text{Carry}_{R5} + C6)$
- **For R7:** $FA_{R7} = FA(\text{Carry}_{33}, \text{Carry}_{R6}, C7)$ → $R7 = \text{Sum}$

⚠ Implementation Note for VCB (Virtual Circuit Board) When generating the final bits (R5,R6,R7), you will encounter instances where three or four distinct signals collide (for example: Carry_{13} , Carry_{R4} , and $C5$).

To handle this cleanly without logic race conditions, simply **cascade two Full Adders in series** for that specific bit column. This guarantees all inputs are fully resolved into a proper sum and a single carry-out bit.